

WorkHub Phase 1 Design Note

1. Architecture Overview

WorkHub follows a clean layered Spring Boot architecture:

- Controller layer: REST endpoints, request validation, response mapping.
- Service layer: business logic, tenant validation, transaction boundaries.
- Repository layer: JPA persistence with tenant-scoped queries.
- Security layer: JWT authentication, request filter chain, tenant context extraction.

Core packages:

- `com.workhub.controller`
- `com.workhub.service`
- `com.workhub.repository`
- `com.workhub.security`
- `com.workhub.dto`
- `com.workhub.exception`

2. Tenant Isolation Approach

The project uses shared database with tenant discrimination via `tenant_id`:

- JWT includes `tenantId` claim after login.
- `JwtAuthenticationFilter` extracts identity and tenant context per request.
- `TenantContext` stores current tenant in thread-local scope.
- Service methods call `TenantContext.getRequiredTenantId()`.
- Repository methods filter by tenant (`findByIdAndTenantId`, `findByTenantId`, etc.).
- Cross-tenant access is rejected with consistent error responses.

This enforces “Tenant A cannot read/write Tenant B data” at service + query level.

3. Authentication Flow (JWT)

- POST `/api/auth/login` validates credentials via Spring Security authentication manager.
- On success, server returns JWT containing:
 - `sub` (email)
 - `userId`
 - `tenantId`
 - `role`
- GET `/api/auth/me` requires JWT and returns authenticated user profile and tenant information.

Security policy:

- Public: `/api/auth/login`, `/api/health`, `/error`
- Protected: all other endpoints including `/api/auth/me`

4. Validation + Error Handling

Validation is applied using Jakarta Bean Validation (`@Valid`, `@NotBlank`, `@Email`, `@Size`, etc.) on DTOs.

`GlobalExceptionHandler` (`@ControllerAdvice`) standardizes JSON errors:

- Validation errors (400)
- Not found / invalid arguments (404/400)
- Forbidden tenant/resource access (403)
- Unauthorized/authentication errors (401)
- Unexpected server errors (500)

Response format:

- `message`
- `details` (list)

5. Transaction Boundary and Rollback Demonstration

Phase 1 transactional demonstration is implemented by:

- Service: `TransactionalTestService#testTransactionRollback`
- Controller: `TransactionTestController`

Flow:

1. Begin transaction (`@Transactional`).
2. Create project.
3. Create task linked to project.
4. Throw intentional runtime exception.
5. Entire transaction rolls back.

Verification endpoint:

- GET `/api/test/transaction/verify?projectId=X&taskId=Y`

Expected outcome:

- Both records do not exist after rollback.

6. Phase 1 API Coverage

Implemented Phase 1 required endpoints:

- POST `/api/auth/login`
- GET `/api/auth/me`

- `POST /api/projects`
- `GET /api/projects`
- `GET /api/projects/{id}`
- `POST /api/projects/{id}/tasks`
- `PATCH /api/tasks/{id}`

7. Operational Notes

- Health endpoint: `GET /api/health`.
- Test profile (`application-test.yml`) uses in-memory H2 DB for CI-safe tests.
- Demo flow script is available at `scripts/phase1-curl-demo.sh`.

8. Build/Test Result

- `mvn test` passes using the `test` profile.
- Application context loads successfully with schema creation and seed data.